

00 Math

Activation Functions Evolution

The Big Picture

Every new activation function was invented to solve a limitation of the previous one:

```
Linear
  ↓ No non-linearity
Sigmoid
  ↓ Vanishing gradients, not zero-centered
Tanh
  ↓ Still saturates
ReLU
  ↓ Dying neurons
Leaky ReLU
  ↓ Hard threshold, not smooth
GELU
  ↓ Higher computational cost
SiLU (Swish)
```

1. Linear Activation

$$y = Wx + b$$

Problem: Stacking multiple linear layers produces another linear transformation.

```
Layer 1:  y = W1x + b1
Layer 2:  z = W2y + b2
Substitute: z = W2(W1x + b1) + b2 = (W2W1)x + (W2b1 + b2) = Wx + b
```

100 linear layers = 1 linear layer. Deep networks gain no expressive power.

Motivation: Neural networks require non-linearity. Without it, deep learning does not exist.

2. Sigmoid

```
Range: 0 → 1
σ(x) = 1 / (1 + e(-x))
```

Advantages

- Smooth and differentiable
- Ideal for binary probability outputs

Problems

- Not zero-centered
- Saturates for large positive/negative inputs
- Causes vanishing gradients

Today mainly used for: binary classification outputs, gates in LSTM/GRU.

3. Tanh

Range: $-1 \rightarrow 1$
 $\tanh(x) = (e^x - e^{-x}) / (e^x + e^{-x})$

Motivation

Improve upon Sigmoid by making activations zero-centered.

Advantages

- Zero-centered outputs
- Better optimization than Sigmoid

Problem

Still saturates for large magnitudes. Vanishing gradients remain.

4. ReLU

$\text{ReLU}(x) = \max(0, x)$

Advantages

- Extremely simple and fast
- No saturation for positive values
- Enabled successful training of deep networks

Problem: Dying ReLU

For negative inputs:

```
Output = 0
Derivative = 0
Weight update:  $w = w - \eta \nabla L$  where  $\nabla L = 0$ 
→ No learning occurs
→ Neuron may permanently output zero
```

Motivation: Can we prevent neurons from dying?

5. Leaky ReLU

```
Instead of: negative → 0
Use:       negative → 0.01x
```

Advantages

- Gradient never becomes exactly zero
- Neurons can recover
- Reduces dying ReLU problem

Remaining Problem

Still has a hard threshold at zero. Not smooth.

Motivation: Can activation become smooth instead of making hard ON/OFF decisions?

6. GELU (Gaussian Error Linear Unit)

Key Idea

Instead of asking "Is x positive?", ask "How likely should this neuron activate?"

Activation becomes **probabilistic** rather than binary.

Characteristics

- Smooth
- Allows partial activation
- Small positive values activate weakly
- Small negative values are not immediately discarded

Advantages

- Better optimization
- More expressive than ReLU
- Excellent for NLP

Used by

BERT, GPT-2, GPT-3

Limitation

More computationally expensive than ReLU.

7. SiLU (Swish)

$$\text{SiLU}(x) = x \cdot \text{sigmoid}(x)$$

Intuition

Input	Behavior
Large positive	Almost unchanged
Large negative	Gradually suppressed
Small values	Partially activated

Advantages

- Smooth everywhere
- No dead neurons
- Excellent gradient flow
- Computationally efficient
- Empirically better for very deep transformers

Used by

Llama, Mistral, Qwen — most modern open-source LLMs.

Evolution Summary

Activation	Innovation	Problem Solved	Remaining Problem
Linear	Basic mapping	—	No non-linearity
Sigmoid	Non-linearity, probability	Linear limitation	Vanishing gradients, not zero-centered
Tanh	Zero-centered	Sigmoid saturation	Still saturates
ReLU	Fast, no positive saturation	Vanishing gradients	Dead neurons
Leaky ReLU	Small negative gradients	Dead neurons	Hard threshold, not smooth
GELU	Smooth probabilistic gating	Hard threshold	Higher compute cost
SiLU	Smooth, efficient, stable	Compute cost	Current state-of-the-art

Interview Takeaways

Function	One-liner
Sigmoid	Best for binary probabilities
Tanh	Zero-centered alternative to Sigmoid
ReLU	Made deep learning practical. Main weakness: dead neurons
Leaky ReLU	Allows small gradients for negative inputs
GELU	Smooth probabilistic gating. Preferred by BERT and GPT
SiLU	Smooth, efficient, stable. Current SOTA for most modern LLMs

One-Line Story

```
Linear → Need non-linearity
  → Sigmoid → Need zero-centered outputs
  → Tanh → Need faster deep training
  → ReLU → Need to avoid dead neurons
    → Leaky ReLU → Need smooth activations
      → GELU → Need smooth with lower compute cost
        → SiLU
```

Calculus for ML

Why Calculus?

Machine learning is optimization. Optimization requires gradients. Gradients come from derivatives. Calculus is the foundation.

Derivatives

Definition

The derivative measures how a function changes as its input changes:

$$f'(x) = \lim_{h \rightarrow 0} [f(x+h) - f(x)] / h$$

Key Rules

Rule	Formula
Power rule	$d/dx[x^n] = n \cdot x^{(n-1)}$
Sum rule	$d/dx[f+g] = f' + g'$
Product rule	$d/dx[f \cdot g] = f' \cdot g + f \cdot g'$
Chain rule	$d/dx[f(g(x))] = f'(g(x)) \cdot g'(x)$
Constant	$d/dx[c] = 0$

Partial Derivatives

When a function has multiple inputs, the partial derivative measures change with respect to one variable while holding others constant:

$$\partial f / \partial x_i = \text{rate of change of } f \text{ w.r.t. } x_i$$

The **gradient** is the vector of all partial derivatives:

$$\nabla f = [\partial f / \partial x_1, \partial f / \partial x_2, \dots, \partial f / \partial x_n]$$

Chain Rule (The Most Important Rule in ML)

Why it matters

Neural networks are compositions of functions. Backpropagation is just the chain rule applied repeatedly.

Intuition

$$x \rightarrow f \rightarrow g \rightarrow h \rightarrow \text{loss}$$

To know how x affects loss , multiply the local derivatives:

$$\partial \text{loss} / \partial x = \partial \text{loss} / \partial h \cdot \partial h / \partial g \cdot \partial g / \partial f \cdot \partial f / \partial x$$

Example: Linear Regression

$$\begin{aligned} y_{\text{pred}} &= X @ w \\ \text{loss} &= (y_{\text{pred}} - y)^2 \end{aligned}$$

Chain rule:

$$\begin{aligned} \partial \text{loss} / \partial w &= \partial \text{loss} / \partial y_{\text{pred}} \cdot \partial y_{\text{pred}} / \partial w \\ &= 2(y_{\text{pred}} - y) \cdot X \end{aligned}$$

Gradient Descent

Core Idea

Move in the direction that **decreases** the loss:

$$w = w - \text{lr} \cdot \nabla L$$

- lr = learning rate (step size)
- ∇L = gradient of loss w.r.t. weights

Learning Rate Intuition

- Too large → overshoot, diverge
 - Too small → painfully slow convergence
 - Just right → smooth descent
-

Optimization Algorithms

See `optimization_algorithms.md` for the full evolution: Batch GD → SGD → Mini-Batch → Momentum → Adam.

Optimization Algorithms: From Gradient Descent to Adam

The Big Picture

Optimization algorithms answer one question: **Once we've computed the gradients using backpropagation, how should we update the model weights?**

The evolution of optimizers is a story where each new optimizer fixes a limitation of the previous one:

```
Batch Gradient Descent
| Too slow
▼
Stochastic Gradient Descent (SGD)
| Too noisy
▼
Mini-Batch Gradient Descent
| Oscillates
▼
Momentum
| One global learning rate
▼
Adam
```

1. Batch Gradient Descent

Idea

Use the **entire training dataset** to compute the gradient.

```
Entire Dataset → Compute Loss → Compute Gradient → Update Weights
```

Only **one update per epoch**.

Update Rule

```
gradient = compute_gradient(all_training_data)
w = w - lr * gradient
```

Advantages

- Very accurate gradient
- Stable updates
- Smooth convergence

Problems

- For 1,000,000 samples, the optimizer waits for all of them before making a single update
 - For modern LLMs trained on trillions of tokens, this is impractical
-

2. Stochastic Gradient Descent (SGD)

Idea

Use **one training sample** at a time.

```
Sample 1 → Update  
Sample 2 → Update  
Sample 3 → Update
```

Update Rule

```
for sample in dataset:  
    gradient = compute_gradient(sample)  
    w = w - lr * gradient
```

Advantages

- Updates happen immediately
- Learns much faster initially
- No need to wait for the entire dataset

Problems

- Every training sample produces a different estimate of the true gradient
 - The optimizer keeps changing direction → training becomes noisy
-

3. Mini-Batch Gradient Descent

This is what **almost every modern deep learning system** uses.

Instead of 1 sample or 1 million samples, use 32/64/128/256/512 samples.

64 Samples → Average Loss → Average Gradient → Update

Why does Mini-Batching work?

Different samples produce different gradients. When averaged, random noise cancels out. The shared signal remains. The resulting gradient is much closer to the true gradient.

GPU Advantage

GPUs process many samples simultaneously. Sending 1 sample to an RTX A6000 wastes most of the GPU. Sending 64 or 256 samples keeps thousands of CUDA cores busy.

Mini-batches therefore improve both:

- Computational efficiency
- Gradient quality

Comparing the Three

Optimizer	Batch Size	Updates per Epoch	Gradient Quality
Batch GD	Entire Dataset	1	Excellent
SGD	1	One per sample	Very Noisy
Mini-Batch GD	32-1024	Many	Good

Two Types of Oscillation

1. Data Oscillation

Occurs because different training samples produce different gradients.

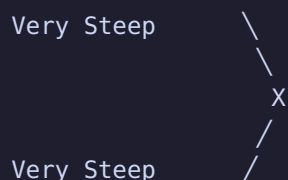
Cat → Gradient →
Dog → Gradient ←
Car → Gradient ↗

Mini-batch gradient descent reduces this by averaging multiple samples.

2. Optimization Oscillation

Even with the perfect gradient, the loss landscape itself may cause oscillation.

Imagine a narrow valley:



Very Steep

Very Steep

x

Gradient Descent repeatedly jumps from one side of the valley to the other. Progress toward the minimum becomes slow.

Momentum was invented to reduce this type of oscillation.

4. Momentum

Problem

Gradient Descent only looks at **today's gradient**. It completely forgets previous updates. This causes zig-zagging in narrow valleys.

Idea

Remember the recent history of gradients. Introduce a variable **velocity** (m) which stores a moving average of past gradients.

Update

```
m = beta1 * m + (1 - beta1) * gradient
w = w - lr * m
```

Intuition

- Gradient Descent asks: "Where should I move today?"
- Momentum asks: "Where have I been moving recently?"

Consistent directions are reinforced. Rapid left-right oscillations are dampened.

5. Adam

Momentum solves oscillation, but still uses **one global learning rate** for every parameter.

Problem

Suppose:

- Weight A gradient = 100
- Weight B gradient = 0.001

Using the same learning rate causes huge updates for Weight A and tiny updates for Weight B. One learning rate does not fit every parameter.

Key Insight

Momentum tells us **where** to move. We also need to know **how large** the gradients usually are.

Instead of storing average gradient, store **average squared gradient**. This is called the **Second Moment**.

Why square the gradients?

Suppose gradients are 100, -100, 100, -100. The mean is 0, which incorrectly suggests small gradients. Squaring gives 10000, 10000, 10000, 10000 — now we correctly know the gradients are consistently large.

Adam stores two statistics

First Moment (average direction):

$$m = \text{beta1} * m + (1 - \text{beta1}) * \text{gradient}$$

Second Moment (average gradient magnitude):

$$v = \text{beta2} * v + (1 - \text{beta2}) * \text{gradient}^2$$

Weight Update:

$$w = w - \text{lr} * m / (\text{sqrt}(v) + \text{eps})$$

Interpretation

- m decides the **direction**
- $\sqrt{v}$ scales the **step size**
- eps prevents division by zero
- Large gradients → Large v → Smaller update
- Small gradients → Small v → Larger effective update

Summary

Optimizer	Solves
Batch GD	Accurate gradients
SGD	Faster updates
Mini-Batch GD	Balances speed and stability
Momentum	Reduces optimization oscillation using gradient history
Adam	Adds adaptive learning rates for every parameter

Interview Takeaways

Why Mini-Batch instead of SGD?

- Reduces noisy gradients
- Better GPU utilization
- Faster convergence

Why Momentum?

- Reduces oscillation in narrow valleys
- Uses past gradients to smooth updates

Why Adam?

- Combines Momentum with adaptive learning rates
- Uses the first moment for direction
- Uses the second moment for scaling the update
- Handles parameters with different gradient magnitudes much better than SGD or Momentum

Linear Algebra

Core Objects

Object	Notation	Shape	Example
Scalar	x	$()$	3.14
Vector	x	$(D,)$	[1.0, -2.3, 0.5]
Matrix	X	(N, D)	batch of embeddings
Tensor	X	(B, N, D)	batch of sequences

- **N** = number of samples
- **D** = number of features / embedding dimension
- **C** = number of classes

Key Operations

Matrix Multiplication

$$(A, B) @ (B, C) = (A, C)$$

Each element is a dot product of a row and a column. Used in every linear layer ($W @ x$), attention ($Q @ K.T$), and embedding lookup.

Dot Product

$$a \cdot b = \sum a_i * b_i$$

Measures similarity. Used in attention scores and cosine similarity.

Matrix Transpose

$$X: (N, D) \rightarrow X.T: (D, N)$$

Swaps rows and columns. Used in gradient computations ($X.T @ \text{gradient}$) and attention ($Q @ K.T$).

Broadcasting

NumPy/PyTorch automatically expands smaller arrays to match larger ones:

```
(N, D) + (D,) → (D,) broadcast to (N, D)
(N, D) * (N, 1) → (N, 1) broadcast to (N, D)
```

Rule: Dimensions must either match or be 1. Broadcasting goes right-to-left.

Shape Discipline

Always ask before writing code:

1. What is the input shape?
2. What is the output shape?
3. What intermediate shapes are needed?

This catches 90% of bugs in ML code.

Complexity

Operation	Time
Dot product	$O(D)$
Matrix-vector	$O(ND)$
Matrix-matrix	$O(N \cdot D \cdot C)$
Attention (Q @ K.T)	$O(N^2D)$

ML operations are often $O(ND)$ because each sample has D features.

Deep Learning Foundations II: BatchNorm, Dropout & Calibration

The Big Picture

After solving optimization and activation functions, researchers focused on three new problems:

```
Unstable Training      → Batch Normalization
Overfitting            → Dropout
Overconfident Predictions → Calibration
```

Each technique solves a completely different problem.

1. Batch Normalization

Motivation

During training, every layer keeps changing its weights. As a result, the distribution of activations seen by the next layer constantly changes.

```
Epoch 1:  [-1.2,  0.8,  1.5, -0.6]
Epoch 20: [  53,   61,   48,   57]
Epoch 40: [-120,  -95, -103, -110]
```

Every layer must continuously adapt to changing input distributions. This is **Internal Covariate Shift**.

Solution

Normalize every feature across the mini-batch:

```
x → Normalize →  $\hat{x}$  →  $\gamma\hat{x} + \beta$ 

where:
   $\gamma$  (gamma) = learnable scaling
   $\beta$  (beta)  = learnable shift
```

Matrix View

	F1	F2	F3
S1	2	5	7
S2	3	8	6
S3	1	4	5
S4	6	7	9

BatchNorm normalizes each **column** (feature) using statistics computed from all samples in the batch.

Advantages

- Faster convergence
- Stable gradients
- Allows larger learning rates
- Mild regularization effect

Problem

BatchNorm depends on multiple samples. During autoregressive inference (batch size = 1), each feature column contains only one value — no meaningful batch statistics can be estimated.

2. LayerNorm (Comparison)

Instead of normalizing columns, LayerNorm normalizes each **row** independently.

```
→→→ 2  5 -1  7  (normalize this token's features)
→→→ 4  8  0  9  (normalize this token's features)
```

Each token is normalized independently using its own feature statistics.

BatchNorm vs LayerNorm

	BatchNorm	LayerNorm
Normalizes	Columns (features)	Rows (samples)
Statistics	Batch statistics	Feature statistics
Batch size	Depends on batch size	Independent of batch size
Best for	CNNs	Transformers

RMSNorm

Observation: Do we really need to subtract the mean? Often no.
Instead, normalize only by the root mean square.

Advantages

- Simpler
- Faster
- Used in Llama and other modern LLMs

Evolution

No Normalization → BatchNorm → LayerNorm → RMSNorm

3. Dropout

Motivation

Neural networks often become overly dependent on a few neurons (**co-adaptation**).



If neuron B learns everything, A and C become lazy. If B fails, the network performs poorly.

Solution

During training, randomly disable neurons.

Before: [2, 5, 1, 8]
After Dropout (50%): [2, 0, 1, 0]

The remaining neurons are forced to learn useful representations. No neuron can assume another neuron will always be available.

Important

Dropout is used **only during training**. During inference, all neurons are active.

Why Scaling is Needed

```
Training: ~50% of neurons survive  
Inference: 100% of neurons active
```

Without correction, activation magnitudes become inconsistent.

Inverted Dropout

Modern implementations scale surviving activations during training:

```
keep_probability = 0.5  
output = output / 0.5
```

This keeps the expected activation approximately the same during training and inference.

Advantages

- Prevents overfitting
- Reduces co-adaptation
- Improves generalization
- Acts as regularization

4. Confidence Calibration

Motivation

A classifier predicts "Cat 99%". But over thousands of examples, predictions with 99% confidence are actually correct only 82% of the time. The model is **overconfident**.

Well-Calibrated Model

Confidence should match reality:

```
Confidence  $\approx$  Accuracy
```

If the model predicts 90% confidence, it should be correct roughly 90% of the time.

5. Temperature Scaling

Question

Can we reduce overconfidence **without retraining**?

Solution

Apply temperature before softmax:

```
Softmax(logits / T)
```

- Higher temperature → softer probabilities → lower confidence
- Prediction (argmax) usually remains unchanged

Two Uses of Temperature

	LLM Sampling	Calibration
Controls	Randomness	Confidence
Higher T	More diverse outputs	Softer probabilities
Same equation	Yes	Yes
Purpose	Creativity	Trustworthiness

6. Label Smoothing

Motivation

Cross entropy trains with `[1, 0]`. The model learns to predict `0.999999` for every training sample. This produces overconfident models.

Solution

Instead of `[1, 0]`, train using `[0.9, 0.1]` or `[0.95, 0.05]`.

The model learns: "This is almost certainly the correct class, but don't become infinitely confident."

Advantages

- Better calibration
- Less overconfidence
- Improved generalization

- Regularization effect

Label Smoothing vs Temperature Scaling

	Label Smoothing	Temperature Scaling
When	During training	After training
Modifies	Labels	Logits
Effect	Prevents overconfidence	Corrects overconfidence
Retraining	Required	Not needed

Summary

Technique	Problem	Solution
BatchNorm	Changing activation distributions	Normalize each feature across the batch
LayerNorm	BatchNorm fails for Transformers / batch=1	Normalize each sample independently
RMSNorm	LayerNorm can be simplified	Normalize by root mean square only
Dropout	Overfitting and co-adaptation	Randomly disable neurons during training
Calibration	Overconfident predictions	Measure whether confidence matches accuracy
Temperature Scaling	Incorrect confidence after training	Scale logits before softmax
Label Smoothing	Cross entropy encourages extreme probabilities	Train using softened target labels

Evolution

Activation Functions

- Need stable optimization → BatchNorm
- Need Transformer-friendly norm → LayerNorm
- Need simpler norm → RMSNorm
- Need regularization → Dropout
- Need trustworthy probabilities → Calibration
- Temperature Scaling
- Label Smoothing

Probability, Information Theory & Statistics

The Big Picture

These three fields answer different questions:

- **Probability:** What is our belief about an outcome before observing it?
 - **Information Theory:** How much do we learn when an event occurs?
 - **Statistics:** How should beliefs change after observing new evidence?
-

1. Probability

Definition

Probability represents our **belief** about an outcome before observing it. It is not the outcome itself.

Fair Die: $P(1) = 1/6$, $P(2) = 1/6$, ..., $P(6) = 1/6$

Properties:

- $0 \leq P(x) \leq 1$
- Sum of probabilities = 1

A collection of probabilities over all outcomes is called a **Probability Distribution**.

2. Information (Surprise)

Question

How much information do we gain when an event occurs?

Observation

- Common events carry **little** information
- Rare events carry **lots** of information

Requirements for an information measure

1. Probability decreases $\rightarrow$ Information increases
2. Certain event ($P=1$) $\rightarrow$ Information = 0
3. Independent events should have **additive** information

Since probabilities multiply while information should add, logarithm naturally appears.

Formula

$$I(x) = -\log(P(x))$$

Interpretation:

- High probability → Low surprise
- Low probability → High surprise

3. Entropy

Question

How surprised are we **on average**?

Entropy is simply the **expected surprise**:

$$H(P) = -\sum P(x) \log(P(x))$$

Interpretation: Average uncertainty of a probability distribution.

Examples

Distribution	Entropy
Fair coin (0.5, 0.5)	High
Biased coin (0.999, 0.001)	Low

Higher entropy means greater uncertainty.

4. Cross Entropy

Entropy measures surprise using the **true** distribution. Cross Entropy measures surprise using the **model's predicted** distribution.

$$H(P,Q) = -\sum P(x) \log(Q(x))$$

Where:

- **P** = True distribution
- **Q** = Model prediction

Interpretation: Average surprise experienced by the model.

In classification with one-hot labels:

```
Loss = -log(predicted probability of the correct class)
```

Why $y * \log(\text{pred})$? The true label acts as a selector. Only the correct class contributes to the loss.

5. KL Divergence

Question

How much **additional** surprise do we experience because our beliefs are incorrect?

```
KL(P||Q) = CrossEntropy(P,Q) - Entropy(P)
```

Interpretation:

```
Cross Entropy = Entropy + KL Divergence
```

Properties:

- $KL \geq 0$
- $KL = 0$ only when $P = Q$

Intuition: KL Divergence is the "cost of having incorrect beliefs."

5b. Mutual Information

Question

Does knowing one variable tell me something about the other?

This is about **variables**, not events.

Example

```
Variable 1: Height  
Variable 2: Weight
```

```
If I tell you someone is 220 cm tall...  
Does that help you guess their weight?  
Yes → Positive mutual information.
```

Another Example

Variable A: Outside Temperature
Variable B: Ice Cream Sales

Hot day → You'd expect higher ice cream sales.
Temperature ↔ Ice Cream Sales → High mutual information.
But they're certainly not mutually exclusive — they happen together!

Mutually Exclusive vs Independent vs Mutual Information

These are three completely different ideas sharing the word "mutual":

Concept	Question
Mutually Exclusive	Can these two events happen together?
Independent	Does one event affect the probability of the other?
Mutual Information	Does knowing one variable reduce uncertainty about the other?

Coin Toss Example

Event A: Heads
Event B: Tails

- **Mutually exclusive?** Yes — they can't both happen on the same toss.
- **Independent?** No — if you know it's Heads, it definitely isn't Tails.
- **Mutual information?** Knowing one completely determines the other, so they share information.

Don't let the word "mutual" fool you — it's just an English word reused in different contexts.

Relationship to KL Divergence

Mutual information $I(X; Y)$ measures how much knowing Y reduces uncertainty about X. It can be expressed as the KL divergence between the joint distribution and the product of marginals:

$$I(X; Y) = KL(P(X, Y) \parallel P(X) \cdot P(Y))$$

If X and Y are independent, $P(X, Y) = P(X) \cdot P(Y)$, so $I(X; Y) = 0$ — knowing Y tells you nothing about X.

6. Perplexity

Cross entropy is difficult to interpret. Researchers convert it into an equivalent number of equally likely choices.

$$\text{Perplexity} = \exp(\text{Cross Entropy})$$

Interpretation: Average number of equally likely choices the model is confused between.

Perplexity	Meaning
2	Model behaves as if choosing between ~2 next tokens
100	Model is highly uncertain

Relationship:

$$\text{Lower Cross Entropy} \rightarrow \text{Lower Perplexity} \rightarrow \text{Better language model}$$

7. Conditional Probability

Definition

Probability of A after learning that B is true.

$$P(A|B) = P(A \cap B) / P(B)$$

Interpretation: Conditional probability **shrinks the universe**.

Example

52 cards → Given card is a Spade → Universe becomes 13 cards

$$P(\text{Ace of Spades} \mid \text{Spade}) = 1 / 13$$

The numerator stays the same. The denominator changes because impossible worlds are discarded.

8. Bayes' Theorem

Derived directly from conditional probability:

$$P(A|B) = \frac{P(B|A) P(A)}{P(B)}$$

Interpretation: Bayes **reverses** probability. It converts:

- "What is the probability of observing the evidence given the cause?"
- into "What is the probability of the cause given the observed evidence?"

Examples

Disease → Symptoms (reality runs cause → effect)
 Fire → Smoke
 Rain → Umbrella

Inference runs: Effect ↑ → Cause?

9. Expectation

Question

What happens **on average**?

$$E[X] = \sum P(x) \cdot x$$

Interpretation: Weighted average of all possible outcomes.

Connection: Entropy itself is simply the expectation of surprise:

$$\text{Entropy} = E[-\log(P)]$$

10. Variance

Expectation tells us the average. Variance tells us **how spread out** values are.

Naively averaging deviations fails because positives and negatives cancel. Solution: **square** deviations.

$$\text{Var}(X) = E[(X - \mu)^2]$$

Interpretation: Average squared distance from the mean. Higher variance means greater spread.

11. Standard Deviation

Variance has squared units (e.g., height variance = cm²). Standard deviation restores the original units.

$$\text{StdDev} = \sqrt{\text{Variance}}$$

Interpretation: Typical deviation from the mean.

12. Covariance

Question

Do two variables move together?

$$\text{Cov}(X, Y) = E[(X - \mu_x)(Y - \mu_y)]$$

Covariance	Meaning
Positive	Both variables increase together
Negative	One increases while the other decreases
Near zero	No linear relationship

Relationship: Variance is simply covariance with itself:

$$\text{Var}(X) = \text{Cov}(X, X)$$

13. Correlation

Problem

Covariance depends on measurement units.

Solution

Normalize covariance:

$$\text{Correlation} = \text{Cov}(X, Y) / (\sigma_x \cdot \sigma_y)$$

Value	Meaning
+1	Perfect positive relationship
0	No linear relationship
-1	Perfect negative relationship

14. Gaussian Distribution

Question

Why do so many natural phenomena follow a bell curve?

Answer

Because many independent random factors combine together. This is explained by the **Central Limit Theorem**.

Examples

Height, IQ, sensor noise, measurement errors, weight initialization.

Parameters

The Gaussian is fully described by only two parameters:

Parameter	Role
Mean (μ)	Determines the center
Standard Deviation (σ)	Determines the width

Notation: $N(\mu, \sigma^2)$

68-95-99.7 Rule

Range	% of values
$\pm 1\sigma$	~68%
$\pm 2\sigma$	~95%
$\pm 3\sigma$	~99.7%

Applications

Diffusion models (Gaussian noise), weight initialization, measurement errors, Kalman filters, Gaussian processes, data normalization.

Key Interview Takeaways

Concept	One-line
Probability	Belief before observing an outcome
Information	Surprise ($-\log(P)$)
Entropy	Average surprise
Cross Entropy	Average surprise using model's predicted probabilities
KL Divergence	Extra surprise caused by incorrect beliefs
Mutual Information	Does knowing one variable reduce uncertainty about the other?
Perplexity	Average number of equally likely choices the model considers
Conditional Probability	Shrink the universe after observing evidence
Bayes' Theorem	Reason backwards from observed effects to possible causes
Expectation	Weighted average
Variance	Average squared deviation from the mean
Standard Deviation	Typical deviation in original units
Covariance	Do two variables move together?
Correlation	Normalized covariance (unitless, -1 to +1)
Gaussian	Distribution produced by many independent random effects